

Define once, stratify for free

Transporting a program along a model morphism

A morphism of models induces a map of targets, and programs can be transported along it. Here a lockdown and a vaccination are written once on SIR and lifted to an age-stratified SIR obtained as a pullback of Petri nets. The lifted program then composes with a stratum-specific refinement, and pushing forward the other way shows when merging strata creates a conflict.

```
using CategoricalInterventions
using AlgebraicPetri
using Catlab.Graphics: to_graphviz
using DiffEqCallbacks
using LabelledArrays
using OrdinaryDiffEq
using Plots

default(; linewidth=2, grid=false)
```

Base model and program

The base net carries reflexive `stay_*` transitions so that the strata's transitions have something to pair with.

```
sir_r = LabelledPetriNet([:S, :I, :R],
    :inf => ((:S, :I) => (:I, :I)), :rec => (:I => :R),
    :stay_S => (:S => :S), :stay_I => (:I => :I), :stay_R => (:R => :R))
base = Model(sir_r)

program = @interventions base begin
    lockdown    = scale(:inf, 0.5; during=5.0..25.0)
    vaccination = transfer(:S => :R, 50.0; at=15.0)
end
```

Program(2 atoms)

Stratify

The type system has one species and three transition types. The age net has four infection transitions (who infects whom), two recoveries, and ageing. `stratify` takes the pullback of the two typing maps and returns the stratified net with its two projections.

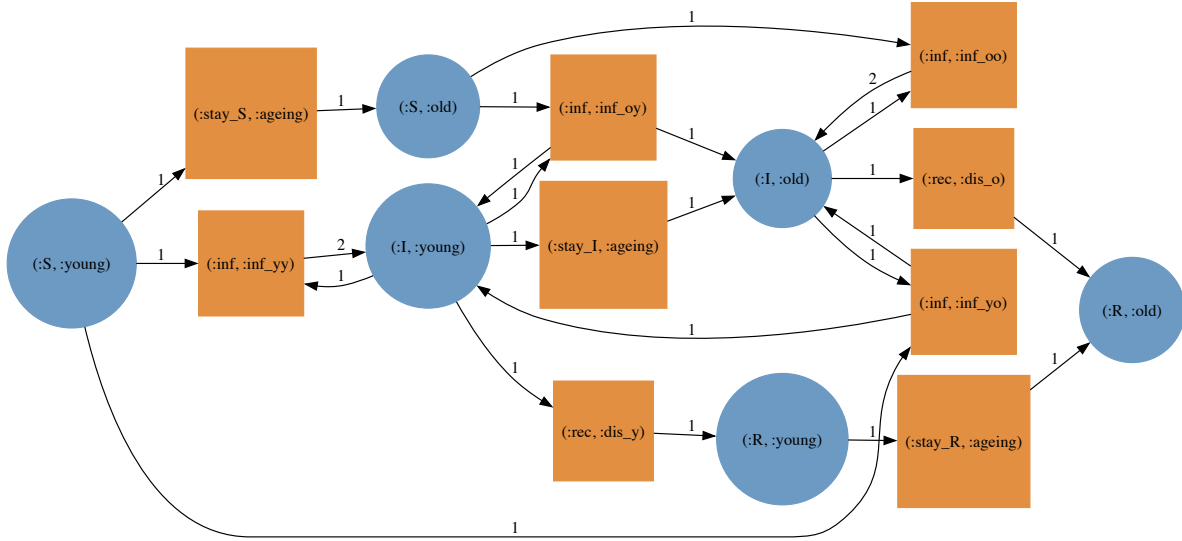
```
types = LabelledPetriNet([:Pop], :infect => ((:Pop, :Pop) => (:Pop, :Pop)),
                        :disease => (:Pop => :Pop), :strata => (:Pop => :Pop))
age = LabelledPetriNet([:young, :old],
  :inf_yy => ((:young, :young) => (:young, :young)),
  :inf_yo => ((:young, :old) => (:young, :old)),
  :inf_oy => ((:old, :young) => (:old, :young)),
  :inf_oo => ((:old, :old) => (:old, :old)),
  :dis_y => (:young => :young), :dis_o => (:old => :old),
  :ageing => (:young => :old))

st = stratify(sir_r, age, types,
  Dict(:inf => :infect, :rec => :disease, :stay_S => :strata, :stay_I => :strata, :stay_R => :strata),
  Dict(:inf_yy => :infect, :inf_yo => :infect, :inf_oy => :infect, :inf_oo => :infect,
    :dis_y => :disease, :dis_o => :disease, :ageing => :strata))

strat = Model(st.net)
(species=states(strat), transitions=parameters(strat))
```

```
(species = [:S_young, :S_old, :I_young, :I_old, :R_young, :R_old], transitions = [:inf_inf_y,
```

```
to_graphviz(st.net)
```



Lift the program

`lift` places a copy of each atom on every stratified target over its base target. The lockdown lands on all four infection transitions; the transfer is matched within each stratum, so young susceptibles go to young recovered.

```
lifted = lift(program, st.to_base)
lifted.atoms
```

6-element Vector{Atom}:

```
lockdown_inf_inf_oo: Parameter:inf_inf_oo scale(0.5) on [5.0, 25.0)
lockdown_inf_inf_oy: Parameter:inf_inf_oy scale(0.5) on [5.0, 25.0)
lockdown_inf_inf_yo: Parameter:inf_inf_yo scale(0.5) on [5.0, 25.0)
lockdown_inf_inf_yy: Parameter:inf_inf_yy scale(0.5) on [5.0, 25.0)
vaccination_S_old: State:S_old transfer(50.0 → R_old) on {15.0}
vaccination_S_young: State:S_young transfer(50.0 → R_young) on {15.0}
```

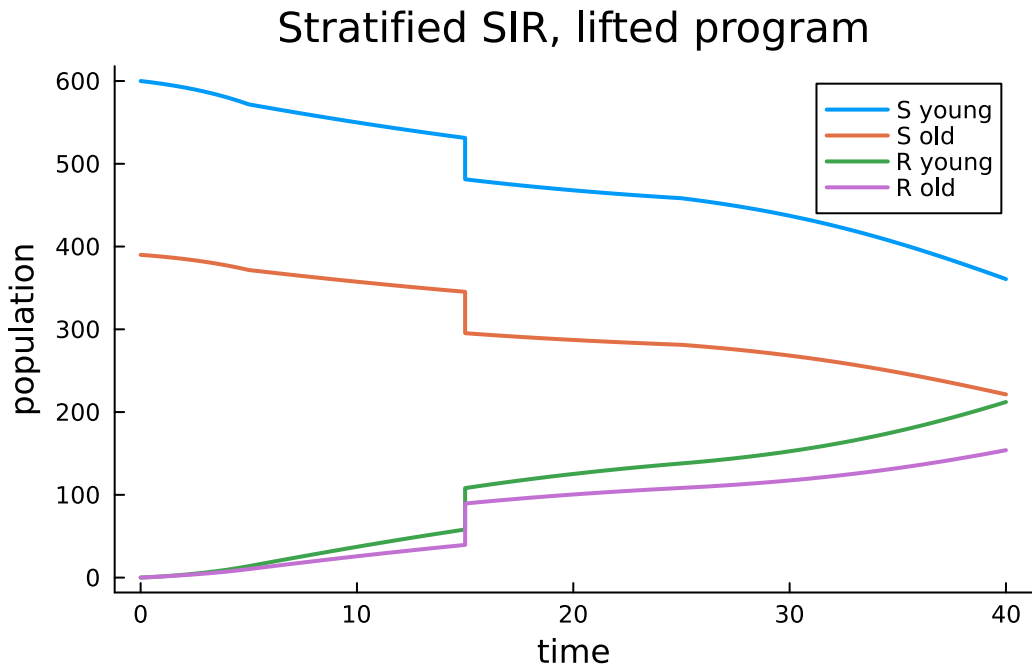
```
u0 = LVector(S_young=600.0, S_old=390.0, I_young=5.0, I_old=5.0, R_young=0.0, R_old=0.0)
rate(t) = startswith(string(t), "inf") ? 0.0005 : startswith(string(t), "rec") ? 0.25 :
          t == :ageing_stay_S || t == :ageing_stay_I || t == :ageing_stay_R ? 0.0 : 0.0
p0 = LVector(; (t => rate(t) for t in parameters(strat))...)
tspan = (0.0, 40.0)

sol = simulate(strat, lifted; u0, p0, tspan, alg=Tsit5(), saveat=0.5)
plot(sol.t, sol[1, :]; label="S young", xlabel="time", ylabel="population", title="Stratified")
```

```

plot!(sol.t, sol[2, :]; label="S old")
plot!(sol.t, sol[5, :]; label="R young")
plot!(sol.t, sol[6, :]; label="R old")

```



The lift is faithful

With identical strata, no ageing, and the transfer split evenly, the stratified model summed over strata must reproduce the base model exactly (up to solver tolerance).

```

u0h = LVector(S_young=495.0, S_old=495.0, I_young=5.0, I_old=5.0, R_young=0.0, R_old=0.0)
u0b = LVector(S=990.0, I=10.0, R=0.0)
p0b = LVector(inf=0.05 * 10.0 / 1000, rec=0.25, stay_S=0.0, stay_I=0.0, stay_R=0.0)
p0h = LVector(; (t => (startswith(string(t), "inf") ? p0b.inf : startswith(string(t), "rec")
                        for t in parameters(strat))...))
half = @interventions base begin
    lockdown    = scale(:inf, 0.5; during=5.0..25.0)
    vaccination = transfer(:S => :R, 25.0; at=15.0)
end
tight = (abstol=1e-10, reltol=1e-10)
sol_h = simulate(strat, lift(half, st.to_base); u0=u0h, p0=p0h, tspan, alg=Tsit5(), saveat=0

```

```
sol_b = simulate(base, program; u0=u0b, p0=p0b, tspan, alg=Tsit5(), saveat=0.5, tight...)
maximum(abs, (Array(sol_h)[1, :] .+ Array(sol_h)[2, :]) .- Array(sol_b)[1, :])
```

2.956653588626068e-9

Refine one stratum

A school closure touches only young-to-young transmission. It composes with the lifted program because it is a further scaling on a set-then-scale target.

```
school = Program(strat, scale(:inf_inf_yy, 0.3; during=8.0..12.0, id=:school))
refined = compose(lifted, school)
[(e.support, e.effects[Target(:inf_inf_yy)]) for e in epochs(refined) if haskey(e.effects, T
```

```
3-element Vector{Tuple{Span{Float64}, CategoricalInterventions.AbstractEffect}}:
 ([5.0, 8.0), scale(0.5))
 ([8.0, 12.0), scale(0.15))
 ([12.0, 25.0), scale(0.5))
```

Push forward, and when merging fails

Pushing a stratified program forward along the projection merges each stratum's targets onto the base target. Two scalings merge into one product; two absolute assignments cannot.

```
two_scalings = Program(strat, scale(:inf_inf_yy, 0.5; during=0.0..10.0, id=:schools),
                        scale(:inf_inf_oo, 0.5; during=0.0..10.0, id=:care_homes))
merged = pushforward(two_scalings, st.to_base)
[(a.id, a.target) for a in merged.atoms], apply(merged, Dict(:inf => 1.0))[1].values[:inf]
```

```
(Tuple{Symbol, Target}[(:schools, Parameter:inf), (:care_homes, Parameter:inf)], 0.25)
```

```
two_sets = Program(strat, set(:inf_inf_yy, 0.1; during=0.0..10.0, id=:a),
                    set(:inf_inf_oo, 0.2; during=0.0..10.0, id=:b))
try
    pushforward(two_sets, st.to_base)
catch err
    sprint(showerror, err)
end
```

```
"Intervention program has 1 conflict(s):\n - Conflict(Parameter:inf on [0.0, 10.0): a vs b;
```

What the category theory bought

Lifting is functorial and preserves conflict-freeness in both directions (`lift_comp`, `conflictFree_lift_iff`), and the lifted program acts on each stratum exactly as the base program acts on the base target (`apply_lift`). Pushing forward along an injective target map preserves conflict-freeness (`conflictFree_pushforward_of_injOn`); the projection is not injective on transitions, so conflicts can appear, and they are reported rather than resolved silently. The stratified net itself is a pullback in the category of Petri nets computed by Catlab.